

the number of documents and n is the number of documents a term t has appeared in.
So, $IDF(This) = \log(2/2) = 0$.

So, what's the best way to explain why IDF exists. Ideally, if a term appears multiple times in a paper, it is unlikely that it is significant to that document. However, if it appears in a subset of papers, it is likely that the word is relevant to the documents in which it appears.

Let us compute IDF for the word 'pandemic':

$IDF(pandemic) = \log(2/1) = 0.301$.

Now, let us compare the TF-IDF for a common word 'this' and a word 'pandemic' which seems to be of relevance to Document 1.

$TF-IDF(this, Document1) = (1/8) * (0) = 0$
 $TF-IDF(this, Document2) = (1/5) * (0) = 0$
 $TF-IDF(pandemic, Document1) = (4/8) * 0.301 = 0.15$

As you can see, for Document 1, the TF-IDF method heavily penalises the word 'this', but assigns greater weight to 'pandemic'. So, this may be understood as 'pandemic' is an important word for Document 1 from the context of the entire corpus.

Bag of words

Humans can understand a language in a fraction of a second. Machines, on the other hand, are unable to comprehend a language. We need to translate the text into a numerical format that the system can interpret. Because most machine

learning and statistical models deal with quantitative data, we must transform all the texts to numbers. Bag of words is one of the simplest techniques to do this. In this approach, we perform two operations:

- Tokenization
- Vectors creation

Tokenization: This is the process of breaking down each sentence into words or smaller chunks. Each word or symbol is referred to as a token in this context. We can take unique words from the corpus after tokenization. The tokens we have from all the documents we're examining for the bag of words construction are referred to as 'corpus'.

Vectors creation: Here, the size of the vector is equal to the number of unique words of the corpus. We can fill each position of a vector with the corresponding word frequency in each sentence. Let's look at an example.

- I like to drink tea.
- Tea is good for your health.
- Tea is more popular than coffee in India.

If we want to analyse the above sentences or build any ML model, we first need to convert them into numbers, as I mentioned above.

The first step is to tokenize the sentences. This is nothing but splitting the above three sentences into individual words.

The next step is to find all the unique words from the above sentences and build the vocabulary. We'll have to make a dictionary with keys and values. The words in our corpus serve as keys, and the frequency of those terms throughout the corpus serves as values. To put it another way, simply count the number of times each word appears in each sentence.

From Table 1, we can see the numbers corresponding to the respective sentences. The first row represents the vector for sentence 1. Given below are the vectors for:

Sentence 1 =
 $[1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$
 Sentence 2 =
 $[0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0]$
 Sentence 3 =
 $[0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1, 0]$

The bag of words paradigm is based on this concept. The above vector representation can be used as an input for ML or statistical models.

Word2vec

In a sentence, every word is dependent on another word or words. Word dependencies must be captured if we are to uncover commonalities and relationships between words.

We can't capture the meaning or relationship of words from vectors using the bag of words and TF-IDF approaches. Word2vec creates embeddings, which are vectors. The word2vec model takes a big corpus as input and outputs it in vector space. The dimensionality of this vector space could be in the hundreds. This vector space will be used to position each word vector.

In vector space, all words in a corpus that are closer to each other share a common context. In this space, a word vector contains the positions of associated words.

Continuous bag of words (CBOW) and skip-gram are two neural network-based versions offered by word2vec. The neural network model in CBOW takes a variety of words as

	I	like	to	drink	tea	is	good	for	health	more	popular	in	India	than	coffee
Sentence 1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0
Sentence 2	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0
Sentence 3	0	0	0	0	1	1	0	0	0	1	1	1	1	1	0

Table 1